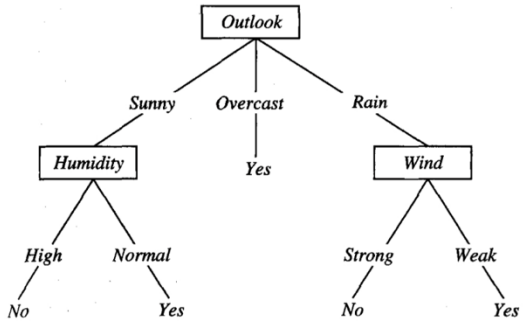
TUGAS II - IF5171 PEMBELAJARAN MESIN DSAI

ARIEF PURNAMA MUHARRAM (23521013)

RESUME DECISION LEARNING TREE

Decision Tree Learning (DTL) merupakan suatu metode untuk memperkirakan fungsi target, di mana fungsi pembelajarannya *(learning rate)* direpresentasikan oleh suatu pohon keputusan *(decision tree)*. Struktur pohon yang terbentuk kemudian dapat dibaca sebagai suatu mekanisme *"if-then-else"* oleh manusia. Algoritma DTL yang sering digunakan yaitu ID3 (Quinlan, 1986), ASSISTANT, dan C4.5 (Quinlan, 1993).



Gambar 1. Contoh *decision tree*

DTL dapat digunakan untuk pendekatan masalah dengan karakteristik sebagai berikut.

- Setiap *instance* direpresentasikan oleh pasangan atribut-nilai *(attribute-value pairs)*.
- Fungsi target yang hendak diestimasikan berupa nilai diskrit.
- *Disjunctive description* mungkin diperlukan.
- Data latih yang digunakan mengandung beberapa kesalahan *(error)*. Hal ini dikarenakan DTL memiliki ketahanan yang lebih baik terhadap terjadinya kesalahan dalam data.
- Data latih mengandung beberapa atribut yang tidak memiliki nilai. DTL dapat digunakan walaupun beberapa atribut tidak memiliki nilai.

Algoritma ID3

Algoritma ID3 *(Iterative Dichotomiser 3)* bekerja dengan membangun pohon dari atas-ke-bawah *(top to bottom)*. Pembentukan pohon dimulai dari atribut yang dapat menjadi *root*. Untuk menentukan atribut yang dapat berperan sebagai *root* maka setiap atribut akan dievaluasi dengan suatu metode statistik. Atribut terbaik kemudian akan dipilih sebagai *root*. Metode statistik yang digunakan tersebut adalah ***information gain***.

ID3(*Examples*, *Target_attribute*, *Attributes*)
Examples are the training examples. Target_attribute is the attribute whose value is to be predicted by the tree. Attributes is a list of other attributes that may be tested by the learned decision tree. Returns a decision tree that correctly classifies the given Examples.

- Create a *Root* node for the tree
- If all *Examples* are positive, Return the single-node tree *Root*, with label = +
- If all *Examples* are negative, Return the single-node tree *Root*, with label = −
- If *Attributes* is empty, Return the single-node tree *Root*, with label = most common value of *Target_attribute* in *Examples*
- Otherwise Begin
  - $A \leftarrow$ the attribute from *Attributes* that best* classifies *Examples*
  - The decision attribute for *Root* $\leftarrow A$
  - For each possible value, v_i, of A,
    - Add a new tree branch below *Root*, corresponding to the test $A = v_i$
    - Let $Examples_{v_i}$ be the subset of *Examples* that have value v_i for A
    - If $Examples_{v_i}$ is empty
      - Then below this new branch add a leaf node with label = most common value of *Target_attribute* in *Examples*
      - Else below this new branch add the subtree ID3($Examples_{v_i}$, *Target_attribute*, *Attributes* − {A})
- End
- Return *Root*

Gambar 2. Algoritma ID3

Information gain bekerja dengan memanfaatkan suatu pengukuran yang disebut sebagai **entropi**. Entropi merupakan suatu pengukuran ketidakmurnian *(impurity)* dari suatu koleksi *(collection)*. Entropi dapat direpresentasikan sebagai berikut.

$$Entropy(S) \equiv \sum_{i=1}^{c} -p_i \, log_2 p_i$$

dimana c adalah jumlah kelas pada koleksi S dan p_i adalah proporsi S yang termasuk ke dalam kelas i. Nilai entropi akan 0 apabila seluruh anggota dari S berada di kelas yang sama.

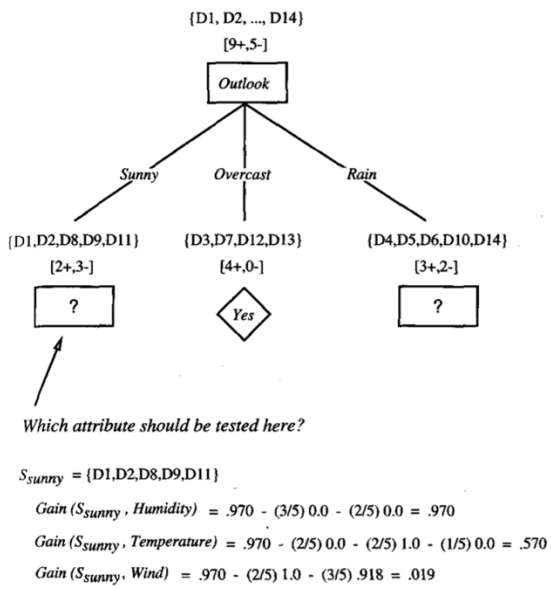
Diketahui entropi merupakan ukuran dari ketidakmurnian dari contoh-contoh latih yang diberikan, selanjutnya kita dapat mengukur seberapa efektif sebuah atribut dapat mengklasifikasikan data. Untuk itu, digunakan *information gain*. Secara sederhana, *information*

gain merupakan suatu reduksi entropi yang disebabkan oleh partisi sampel menurut masing-masing atributnya. *Information gain* dapat dirumuskan sebagai berikut.

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \epsilon Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

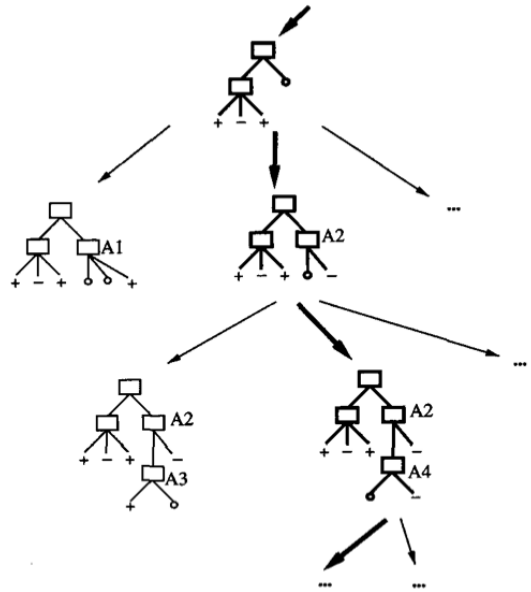
dimana $Values(A)$ merupakan kumpulan dari seluruh kemungkinan nilai untuk atribut A dan S_v merupakan subset dari S dimana atribut A memiliki nilai v (i.e., $S_v = \{s \epsilon S | A(s) = v\}$).

Secara sederhana, bagian pertama pada persamaan *information gain* merupakan entropi awal dari koleksi S, sedangkan bagian kedua menunjukkan entropi dari koleksi S setelah dipartisi menggunakan atribut A. Entropi yang dimaksud pada bagian kedua persamaan merupakan penjumlahan entropi dari setiap subset S_v, yang dibobotkan oleh fraksi sampel $\frac{|S_v|}{|S|}$ yang termasuk ke dalam S_v.



Gambar 3. Proses pembentukan pohon. Atribut *overcast* hanya memiliki satu sampel sehingga dengan demikian langsung membentuk cabang baru. Akan tetapi, dua sisanya perlu dilakukan ekspansi dengan memilih atribut dengan nilai *information gain* tertinggi relatif terhadap subset baru yang terbentuk.

Atribut dengan nilai *information gain* tertinggi akan dipilih sebagai *root*. Proses pemilihan atribut dan partisi akan terus berlanjut sampai terpenuhinya salah satu dari kondisi berikut: (1) setiap atribut telah diikutsertakan sepanjang *path* sepanjang pohon; atau (2) semua nilainya berada pada kelas yang sama (entropinya nol).



Gambar 4. Proses pencarian *hypothesis space* pada algoritma ID3. Algoritma ID3 melakukan pencarian *space* pohon yang memungkinkan dari yang sederhana hingga kompleks, dengan menggunakan informasi dari *information gain*.

Isu pada Algoritma ID3

Beberapa masalah yang perlu diperhatian dalam DTL adalah seberapa dalam cabang pohon yang dapat dibentuk, penanganan atribut dengan nilai kontinu, memilih metode pengukuran atribut yang sesuai, menangani data latih dengan nilai atribut yang hilang *(missing attribute value)*, menangani nilai atribut dengan nilai *cost* yang berbeda, dan meningkatkan efisiensi komputasi. Dalam mengatasi permasalahan tersebut, ID3 telah dikembangkan lebih lanjut dan diberi nama C4.5 (Quinlan, 1993).

Salah satu permasalahan dari DTL adalah *overfitting data*. ***Overfitting data*** merupakan suatu kondisi yang terjadi apabila suatu hipotesis yang dihasilkan memiliki performa baik pada data latih yang diberikan tetapi berperforma kurang baik apabila diberikan data uji.

Overfitting data dapat terjadi akibat data latih yang mengandung kesalahan acak *(random error)* atau *noise*. Data latih yang mengandung kesalah dapat menyebabkan pembentukan cabang pohon yang lebih kompleks. Untuk mengatasi *overfitting* maka dapat dilakukan pendekatan sebagai berikut.

- Menghentikan pembentukan cabang pohon lebih awal, sebelum mencapai titik dimana pohon dapat mengklasifikasikan data latih dengan baik *(stopping early)*.
- Memungkinkan pohon untuk *overfit* terhadap data, kemudian dilakukan pemangkasan cabang pohon setelahnya *(post-prunning)*.

Pendekatan kedua diketahui memiliki hasil yang lebih baik daripada pendekatan pertama dalam menangani *overfitting data*. Hal ini dikarenakan pada pendekatan pertama sulit untuk menentukan pada kedalaman berapa pembentukan pohon perlu dihentikan.

Metode Alternatif Pemilihan Atribut

Terdapat *natural bias* pada metode *information gain* yang lebih memilih atribut dengan jumlah yang paling banyak daripada jumlah yang lebih sedikit. Salah satu cara mengatasi permasalahan tersebut adalah dengan menggunakan metode *gain ratio*. *Gain ratio* memanfaatkan metode *split information* yang sensitif terhadap jumlah dan keberagaman data dalam atribut. *Split information gain* dapat dirumuskan sebagai berikut.

$$SplitInformation(S,A) \equiv -\sum_{i=1}^{c} \frac{|S_i|}{|S|} log_2 \frac{|S_i|}{|S|}$$

Jika diperhatikan pada dasarnya *SplitInformation* merupakan entropi S terhadap nilai-nilai pada atribut A.

Gain Ratio merupakan pengukuran yang dapat didefinisikan sebagai berikut.

$$GainRatio(S,A) \equiv \frac{Gain(S,A)}{SplitInformation(S,A)}$$

Salah satu permasalahan yang dapat muncul dari penggunaan *gain ratio* adalah penyebut yang mungkin nol atau sangat kecil ketika $|S_i| \approx |S|$ untuk salah satu dari S_i. Hal ini menyebabkan *gain ratio* memiliki nilai yang tidak terdefinisi atau sangat besar. Salah satu cara untuk menghindari permasalahan tersebut yaitu dengan terlebih dahulu menghitung *gain* dari setiap atribut, kemudian lakukan perhitungan *gain ratio* hanya pada atribut-atribut yang memiliki nilai di atas rata-rata *gain* (Quinlan, 1986).

DAFTAR PUSTAKA

1. Chapter 3 Mitchell, T., Machine Learning, 1997, McGraw-Hill.